

MbW Lathe System Upgrade — Project Progress Report

Project: MbW Lathe CNC Controller Upgrade (Arduino Mega → Raspberry Pi) **Report Date:** 31 March 2026
Overall Status: 🔄 In Progress

Executive Summary

The project involves replacing the existing Arduino Mega-based lathe controller with a Raspberry Pi 4 running a modern Python-based control system with a full capacitive touchscreen HMI. The new system replicates all functionality of the original MbW firmware (handwheel control, half-nut electronic feed, threading, profile cutting, radius turning) while adding a significantly improved user interface and expandability.

Software development is approximately **60% complete**. The Raspberry Pi 4 and touchscreen display have been ordered. All remaining hardware is yet to be procured.

Overall Progress

Domain	Status	Completion
Software – Core Motion Logic	☑ Complete & Tested	100%
Software – Machine Modes	☑ Complete	100%
Software – HAL (Hardware Abstraction)	☑ Complete	100%
Software – Unit Tests	☑ 55/55 Passing	100%
Software – User Interface (HMI)	🔧 In Development	~50%
Hardware – Procurement	🔄 Partially Ordered	15%
Hardware – Assembly & Wiring	⌚ Not Started	0%
System Integration & On-Machine Testing	⌚ Not Started	0%
Documentation	🔧 In Progress	65%

Overall Project Completion: ~40% **Software Completion: ~60%**

Hardware Status

Ordered ☒

Item	Model	Status
Single-Board Computer	Raspberry Pi 4 Model B (4 GB RAM)	Ordered
Touchscreen Display	Elecrow 7" IPS HDMI 800×480 Capacitive Touch	Ordered

Pending Order ⌚

Item	Model	Priority
microSD Card (32 GB Class 10 / A1)	Samsung Endurance Pro	High
Power Supply – RPi	Official RPi 4 USB-C 5.1V 3A	High
Level Shifters (3.3V → 5V)	TXS0108E module × 2	High
ADC Module	Adafruit ADS1115 (I2C)	High
Z-Axis Encoder	CUI AMT103-V	Medium
X-Axis Encoder	CUI AMT103-V	Medium
Encoder Cables	CUI 435-1FT × 2	Medium
Servo Power Supply	70 V DC, 7 A	Medium
Limit Switches	Omron D2HW-C213MR × 4	Medium
Half-Nut Lever Switch	Omron D2HW-C213MR × 1	Medium
E-Stop Button	Latching NC/NO × 1	High
Push Buttons (momentary)	2-pin momentary × 3	Low
Potentiometer	Bourns 3547S-1AA-103A (10 kΩ)	Medium
Motor Signal Cables	Teknic CPM-CABLE-CTRL-MU120 × 2	Medium

Note: The two ClearPath SDK servo motors are existing hardware from the current machine — no new purchase required.

Software Status

☒ Completed Modules

Hardware Abstraction Layer (**hal/**)

- **mock_interface.py** — Full software simulation, runs on Windows/Linux without hardware. Used for all current testing.
- **rpi_interface.py** — Real Raspberry Pi backend using **pigpio** for hardware-timed step generation.
- **hardware_interface.py** — Abstract base class defining the HAL contract.

Core Motion Engine (**core/**)

- **motion_controller.py** — Z/X handwheel following, velocity banding, autostop logic (ported and improved from Arduino **20_HandWh_Z.ino** / **21_HandWh_X.ino**).
- **spindle.py** — Spindle RPM calculation from index pulse (AutoTech C3 sensor).
- **halfnut.py** — Electronic half-nut feed (ported from **30_HalfNut.ino**).
- **feed_calculator.py** — Potentiometer ADC reading → mm/min feed rate.
- **state_manager.py** — Central **MachineState** dataclass shared across all subsystems.

- `config.py` — All machine parameters in one place (steps/rev, pitch, axis limits, etc.).

Machine Modes (`modes/`)

- `standard_mode.py` — DRO + handwheel mode (default on boot).
- `threading_mode.py` — Synchronized thread cutting with built-in thread table (17 standard sizes: M3–M24, UNC/UNF, imperial).
- `profile_mode.py` — Multi-point profile and taper cutting.
- `radius_mode.py` — Arc and sphere turning.
- `base_mode.py` — Abstract base class for all modes.

Test Suite (`tests/`)

- `test_motion.py` — Motion controller unit tests (autostop, velocity banding, limits).
- `test_halfnut.py` — Half-nut feed tests.
- `test_spindle.py` — RPM calculation tests.
- `test_feed_calculator.py` — Feed rate mapping tests.
- `test_standard_mode.py` — Standard mode integration tests.
- `test_threading_mode.py` — Threading mode tests.
- `simulation.py` — Hardware-free simulation harness.

Test result (last run: 31 March 2026): ☒ 55/55 tests passing

In Progress — User Interface (`ui/`)

The HMI is being actively developed. All screens have been scaffolded with correct layout and navigation. Visual refinement and real-time data binding are ongoing.

Screen	File	Status
Main DRO Screen	<code>screens/main_screen.py</code>	 In Development
Mode Selection	<code>screens/mode_select_screen.py</code>	 In Development
Threading Wizard	<code>screens/thread_screen.py</code>	 In Development
Profile Wizard	<code>screens/profile_screen.py</code>	 In Development
Radius Wizard	<code>screens/radius_screen.py</code>	 In Development
App Orchestrator	<code>app.py</code>	 In Development
Industrial Dark Theme	<code>theme.py</code>	☒ Complete

Target look: Industrial dark theme, 48px monospaced axis readouts (cyan/green/amber), large touch-friendly buttons (minimum 80px height), designed for 7" 800×480 display.

Documentation Status

Document	Status
----------	--------

Document	Status
HARDWARE.md — Component list, GPIO pinout table, wiring ASCII schematic	☑ Complete
README.md — Setup, installation, and run guide	☑ Complete
PROJECT_PROGRESS.md — This document	☑ Created 31 Mar 2026
Wiring PDF / schematic (KiCad or Fritzing)	⌚ Not Started
User Manual (operator guide)	⌚ Not Started

Remaining Work

Software (remaining ~40%)

1. **UI polish & real-time data binding** — Connect [MachineState](#) values to live screen widgets (DRO values, RPM display, mode indicators).
2. **Touchscreen input handling** — Button tap debounce, swipe gestures, on-screen keypad for parameter entry.
3. **Alarm / fault display** — Limit switch triggered overlay, E-stop state, servo fault (HLFB).
4. **End-to-end integration testing** — Run all modes under the simulation harness with realistic encoder playback.
5. **RPi on-device testing** — Deploy to actual Raspberry Pi once hardware arrives; validate [rpi_interface.py](#) with pigpio.

Hardware (remaining ~85%)

1. Order all pending components listed above.
2. Design and build control enclosure / panel.
3. Wire RPi to servo STEP/DIR/ENABLE through level shifters.
4. Wire encoders, limit switches, half-nut switch, E-stop.
5. Connect ADS1115 ADC via I2C for potentiometer.
6. Power supply wiring and fusing.

Integration & Commissioning

1. Deploy software to RPi; run smoke tests.
2. Bench test with motors powered (no load).
3. On-machine test at low feed rates.
4. Full function test: all modes, threading sync, limit switch recovery.
5. Operator acceptance.

Risks & Notes

Risk	Mitigation
RPi GPIO 3.3V vs ClearPath 5V TTL	Level shifters (TXS0108E) are mandatory — included in pending order list

Risk	Mitigation
RPi has no onboard ADC	ADS1115 I2C module required for potentiometer — included in pending order list
Spindle index pulse voltage (AutoTech C3 outputs 5V)	10kΩ/20kΩ voltage divider required between C3 and RPi GPIO
UI not yet deployable on RPi	Mock interface allows continued software testing on Windows until hardware arrives
Threading synchronization accuracy	Core timing relies on pigpio hardware waveforms — must be validated on actual RPi

Last updated: 31 March 2026